Guía de Diagramas con Mermaid

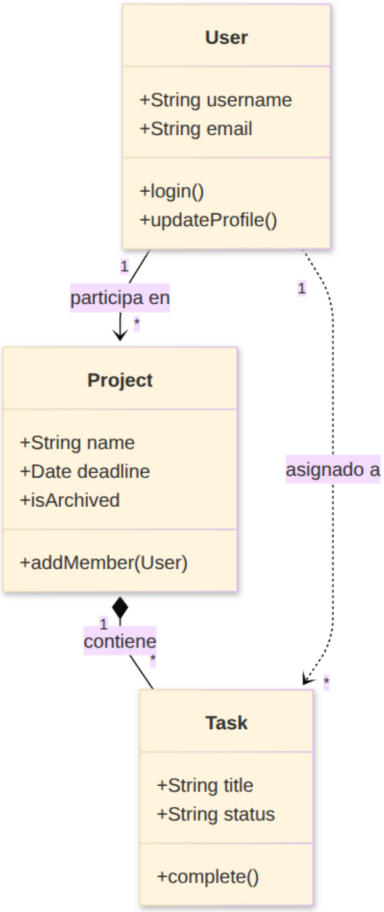
A continuación, tenéis los tipos de diagramas más relevantes para vuestra memoria y cómo implementarlos.

Diagrama de Clases (Class Diagram)

Uso: Representar la estructura estática del sistema. Define los objetos, sus atributos (datos), sus métodos (comportamiento) y cómo se relacionan entre sí. **Cuándo usarlo:**

- **Frontend:** Para visualizar interfaces de TypeScript o componentes complejos.
- **Backend:** Para definir Modelos, Servicios o Controladores.
- **Fullstack:** Para mostrar la lógica de negocio independientemente de cómo se guarde en la base de datos.

Ejemplo: Sistema de Gestión de Tareas Este diagrama muestra que un Usuario puede tener muchas Tareas, y una Tarea pertenece a un Proyecto.




```
classDiagram
    class User {
        +String username
        +String email
        +login()
        +updateProfile()
    }

    class Project {
        +String name
        +Date deadline
        +isArchived
        +addMember(User)
    }

    class Task {
        +String title
        +String status
        +complete()
    }

    %% Relaciones
    User "1" --> "*" Project : participa en
    Project "1" *-- "*" Task : contiene
    User "1" ..> "*" Task : asignado a
```

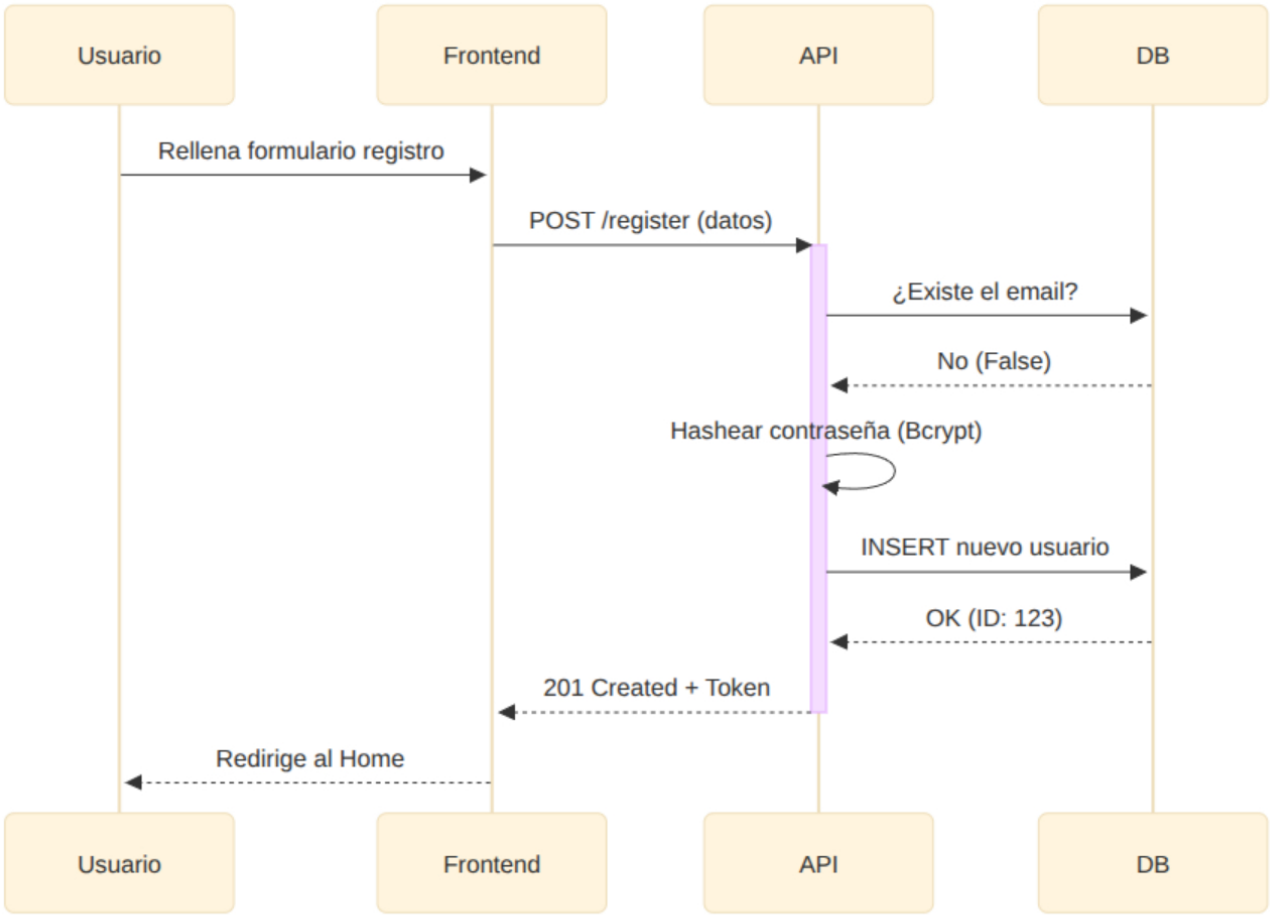
Sintaxis Clave:

- `+` : Público (accesible desde fuera).
- `-` : Privado (uso interno de la clase).
- `-->` : Asociación simple (tiene/usa).
- `*--` : Composición (si borras el Proyecto, se borran las Tareas).
- `"1"` y `"*"` : Cardinalidad (Uno a Muchos).

Diagrama de Secuencia (Sequence Diagram)

Uso: Representar la interacción y el orden de los mensajes entre las partes del sistema a lo largo del tiempo. **Cuándo usarlo:** Imprescindible para explicar flujos complejos como **Login/Auth**, **Pasarelas de Pago** o **Llamadas a APIs externas**.

Ejemplo: Flujo de Registro de Usuario

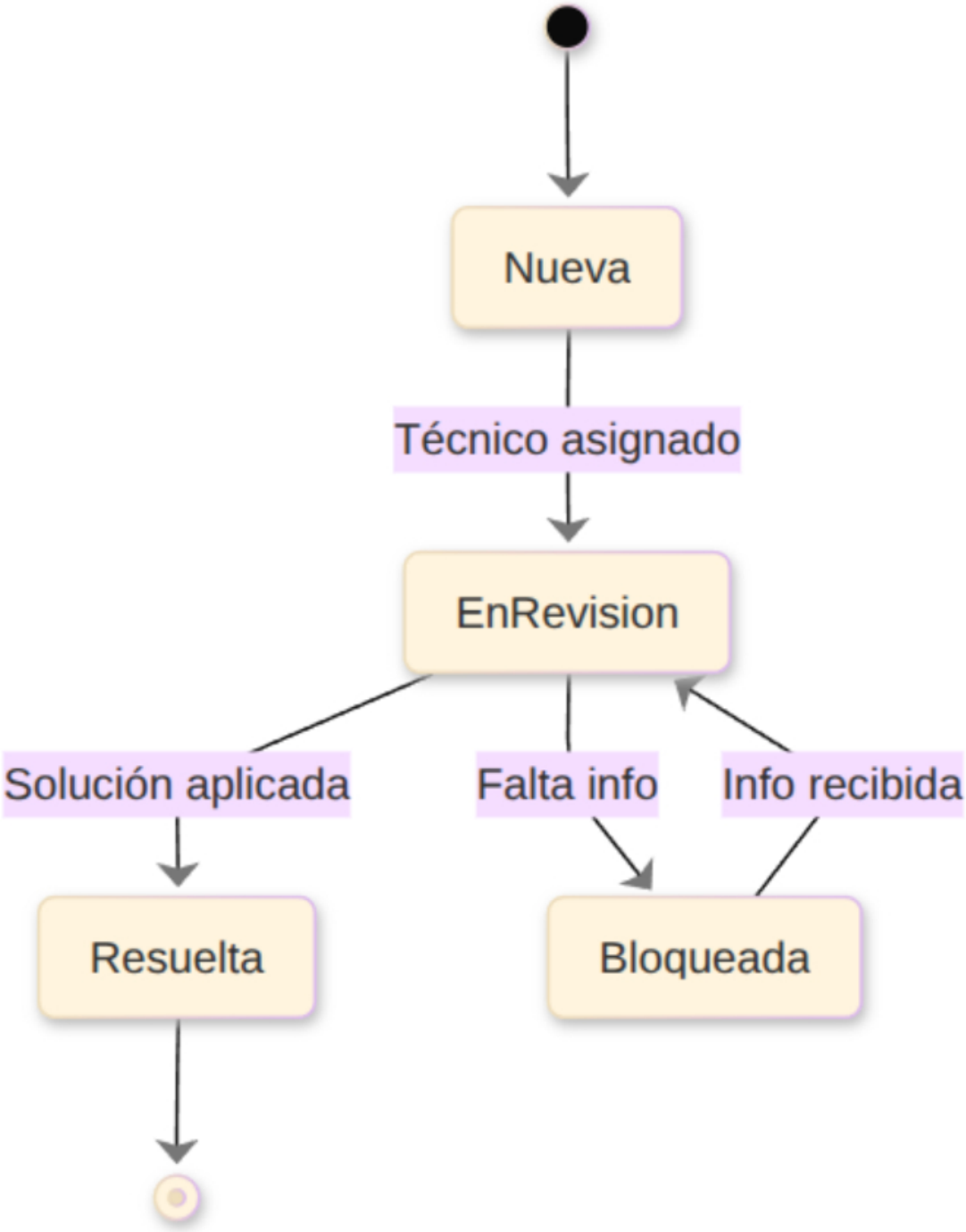



```
sequenceDiagram
    participant U as Usuario
    participant F as Frontend
    participant A as API
    participant D as DB

    U->>F: Rellena formulario registro
    F->>A: POST /register (datos)
    activate A
    A->>D: ¿Existe el email?
    D-->>A: No (False)
    A->>A: Hashear contraseña (Bcrypt)
    A->>D: INSERT nuevo usuario
    D-->>A: OK (ID: 123)
    A-->>F: 201 Created + Token
    deactivate A
    F-->>U: Redirige al Home
```

Diagrama de Estado (State Diagram)

Uso: Mostrar el ciclo de vida de un elemento que cambia de estado. **Cuándo usarlo:** Si vuestro proyecto tiene elementos como **Pedidos** (Pendiente -> Pagado -> Enviado), **Incidencias** (Abierta -> En proceso -> Resuelta) o **Usuarios** (Activo -> Baneado).

Ejemplo: Ciclo de vida de una Incidencia

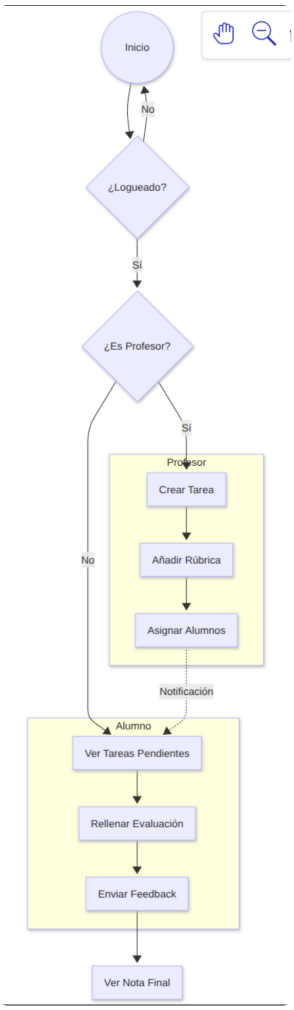


```
stateDiagram-v2
    [*] --> Nueva
    Nueva --> EnRevision: Técnico asignado
    EnRevision --> Resuelta: Solución aplicada
    EnRevision --> Bloqueada: Falta info
    Bloqueada --> EnRevision: Info recibida
    Resuelta --> [*]
```

Diagrama de Flujo (Flowchart)

Uso: Representar algoritmos, lógica condicional ("If/Else") o navegación del usuario. **Cuándo usarlo:** Para explicar una pieza de lógica específica donde hay toma de decisiones.

Ejemplo: Flujo aplicación de evaluación de trabajos

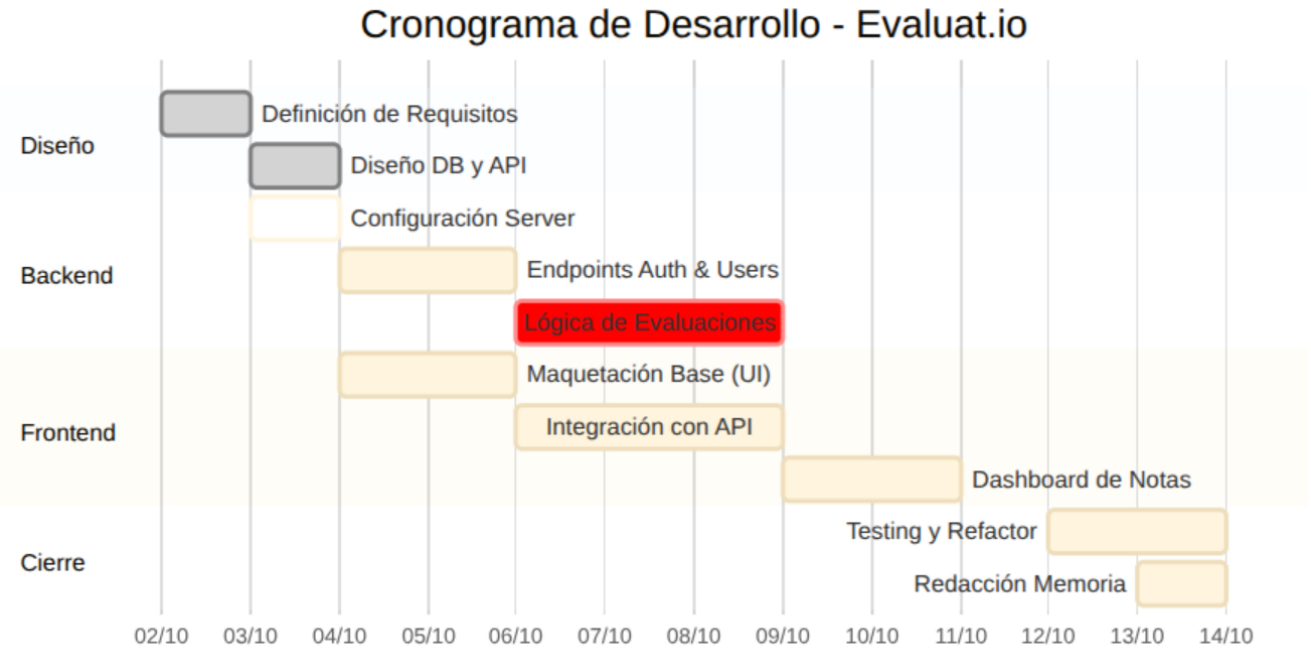


```
graph TD
    %% Nodos
    Start((Inicio))
    Login{¿Logueado?}
    Rol{¿Es Profesor?}

    subgraph Profesor
    CreateTask[Crear Tarea]
    AddRubric[Añadir Rúbrica]
    Assign[Asignar Alumnos]
    end

    subgraph Alumno
    Dashboard[Ver Tareas Pendientes]
    EvalForm[Rellenar Evaluación]
    Submit[Enviar Feedback]
    end

    ViewGrades[Ver Nota Final]

    %% Relaciones
    Start --> Login
    Login -- No --> Start
    Login -- Sí --> Rol

    Rol -- Sí --> CreateTask
    CreateTask --> AddRubric
    AddRubric --> Assign

    Rol -- No --> Dashboard
    Assign -.->|Notificación| Dashboard
    Dashboard --> EvalForm
    EvalForm --> Submit
    Submit --> ViewGrades
```

Diagrama de Gantt (Planificación)

Uso: Visualizar el cronograma del proyecto. **Cuándo usarlo:** En el apartado de "Metodología" o "Planificación" para demostrar organización temporal durante el sprint.

Ejemplo: Cronograma desarrollo de aplicación




```
gantt
    title Cronograma de Desarrollo - Evaluat.io
    dateFormat  YYYY-MM-DD
    axisFormat  %d/%m

    section Diseño
    Definición de Requisitos   :done, des1, 2023-10-02, 1d
    Diseño DB y API            :done, des2, 2023-10-03, 1d

    section Backend
    Configuración Server       :active, back1, 2023-10-03, 1d
    Endpoints Auth & Users     :back2, after des2, 2d
    Lógica de Evaluaciones     :crit, back3, after back2, 3d

    section Frontend
    Maquetación Base (UI)      :front1, after des2, 2d
    Integración con API        :front2, after back2, 3d
    Dashboard de Notas         :front3, after back3, 2d

    section Cierre
    Testing y Refactor         :final1, 2023-10-12, 2d
    Redacción Memoria          :final2, 2023-10-13, 1d
```